In [ ]:

Jeremy Barton
Term Project
DSC550-T302
Spring 2026

WigglNET: Niche Market Data Mining and Brokerage Service

Milestone 1: Data Selection and EDA

IMPORTANT NOTE: This project was reframed to validate the effectiveness of the RoBERTa model rather than to predict the accuracy of its outputs. Ground-truth data is needed for prediction but is not immediately available. In practice, it will be collected through client outcomes, feedback, and business results.

Business Question

Small businesses and enterprises across the globe are looking to enhance their AI models with training data. This can be collected through a "Golden Set" created by recently unemployed human reviewers, which is fast but expensive, or through business outcomes and earnings results, which is slower but more reliable. Another way to improve a model is to continuously measure the effectiveness of the model itself, not just its ability to produce the correct output.

For this project, the hypothetical situation is: **a small business needs to improve the effectiveness of a model used to predict the sentiment of entities and keywords in a specific product category**.

A sentiment analysis model called RoBERTa, developed by Facebook, applies labels to keywords and entities based on perceived context. The reference data comes from various social media platforms, like Twitter.

While the RoBERTa model applies sentiment labels to keywords and entities that may be related to the target market, those labels cannot immediately be treated as truth. They are model-generated classifications based on patterns learned during training, not independently verified ground-truth labels.

Without human review, client outcomes, or business results to validate the labels, the project must treat RoBERTa sentiment as a prediction to evaluate rather than an unquestioned fact.

In practice, ground-truth data used to improve RoBERTa would be realized through client outcomes, manual review, or business results. This would happen after the client received the report of keywords and entities labeled by RoBERTa. WigglNET would then work with the client to monitor outcomes based on the decisions informed by RoBERTa, confirming or denying the labels produced by the model.

Therefore, the question being asked in this business problem must be: ***"Can we validate the sentiment labels produced by RoBERTa for a specific product market niche?"*** Doing so would give extra reassurance to the initial pattern-based report provided to clients.

Exploring the Data

A static CSV file contains the results of a query that pulls combined article, keyword, entity, and sentiment information by joining four other tables on either ARTICLE_ID or KEYWORD_ID. Because this is time-series data, the query can ignore all rows where PUBLISHED_DATE is NULL.

It includes the CONFIDENCE field from ARTICLE_ENTITY_SENTIMENT, which indicates the RoBERTa model's confidence level (0-1) in the sentiment classification for each entity.

In [1]:
```
# Imports and Processing
import sys
import pandas as pd
df = pd.read_csv('keywords.csv')
```

In [2]:
```
# Display basic info about the dataset
print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
```

```
Dataset shape: (19367, 9)

Column names: ['ARTICLE_ID', 'TITLE', 'PUBLISHED_DATE', 'KEYWORD_TEXT', 'ENTITY_TEXT', 'ENTITY_TYPE', 'SENTIMENT', '
CONFIDENCE', 'MENTIONS']
```

Predictor Variables

To validate RoBERTa sentiment labels, the model will use the following predictor variables.

- ENTITY_TYPE
- KEYWORD_TEXT
- ENTITY_TEXT
- TITLE

- PUBLISHED_DATE

IMPORTANT NOTE: `ENTITY_TYPE` , `KEYWORD_TEXT` , and `ENTITY_TEXT` are produced by an upstream Named Entity Recognition (NER) model. NER will be evaluated with the same rigor as RoBERTa, but under a separate workstream. The current project is scoped exclusively to RoBERTa.

How many articles are in this dataset?

In [3]:
```
total_articles = df["ARTICLE_ID"].nunique()
print(f"There are {total_articles} articles in this dataset.")
```

```
There are 132 articles in this dataset.
```

If there are only 132 articles, why are there 19,637 rows?

This dataset includes metadata about keywords and entities in an article, not just a list of each article. Every keyword has entity text, an entity type, a sentiment label, and a number of mentions.

What are the max and min published dates?

In [4]:
```
max_pub = df["PUBLISHED_DATE"].max()
print(f"Max published date is {max_pub}")

min_pub = df["PUBLISHED_DATE"].min()
print(f"Min published date is {min_pub}")
```

```
Max published date is 2026-04-10
Min published date is 2022-04-01
```

Identifying Entities with Positive or Negative Sentiment, Ordered by Negative Sentiment

This analysis focuses on entities with strong sentiment signals (positive or negative), excluding purely neutral entities, and orders them by negative sentiment to highlight problematic entities first. A client might use this graph to understand what brands or keywords to avoid in future business decisions.

Ordering by negative mentions (rather than total volume) surfaces entities that would otherwise be hidden under high-volume neutral coverage, making it the more useful view for a brokerage service flagging risk.

Data Preparation

In [5]:
```
# Import visualization libraries
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('ggplot')

# Sentiment color scheme and stack order (reused across charts)
colors = {
    'negative': '#e74c3c',  # Red
    'neutral':  '#95a5a6',  # Gray
    'positive': '#2ecc71'   # Green
}
sentiment_order = ['negative', 'neutral', 'positive']
```

Aggregate by entity and sentiment, then identify entities that have *any* positive or negative sentiment (filtering out purely neutral entities). Sort by negative mention count to surface the highest-risk entities first.

In [6]:
```
entity_sentiment = df.groupby(['ENTITY_TEXT', 'SENTIMENT'])['MENTIONS'].sum().reset_index()

# find entities that have positive OR negative sentiment (not just neutral)
entity_sentiment_set = df.groupby('ENTITY_TEXT')['SENTIMENT'].apply(lambda x: set(x))
                                                              # convert each groups sentiment values to a Python set

# filter to entities that have 'positive' or 'negative' in their sentiment set
entities_with_sentiment = entity_sentiment_set[
    entity_sentiment_set.apply(lambda x: 'positive' in x or 'negative' in x)
].index.tolist() # convert to list

print(f"Found {len(entities_with_sentiment)} entities with positive or negative sentiment")
```

```
Found 399 entities with positive or negative sentiment
```

In [7]:
```
# Get negative mention counts for these entities
negative_mentions = df[
    (df['ENTITY_TEXT'].isin(entities_with_sentiment)) &
    (df['SENTIMENT'] == 'negative')
].groupby('ENTITY_TEXT')['MENTIONS'].sum()
```

In [8]:
```
# sort by negative mentions (descending) and get top 10
all_sentiment_entities = pd.Series(0, index=entities_with_sentiment)
all_sentiment_entities.update(negative_mentions)
top_entities_by_negative = all_sentiment_entities.nlargest(25).index.tolist()
```

```
# filter to only include selected entities
entity_sentiment_top_negative = entity_sentiment[entity_sentiment['ENTITY_TEXT'].isin(top_entities_by_negative)]
```

Visualization

In [9]:
```
# Pivot the data so sentiments become columns (same logic as first viz)
pivot_entity_negative = entity_sentiment_top_negative.pivot(index='ENTITY_TEXT',
                                                            columns='SENTIMENT',
                                                            values='MENTIONS').fillna(0)

# Reorder columns to control the stack order
pivot_entity_negative = pivot_entity_negative.reindex(columns=sentiment_order, fill_value=0)

# Sort by NEGATIVE mentions (descending) - key difference from first visualization
pivot_entity_negative = pivot_entity_negative.sort_values('negative', ascending=False)
```

In [10]:
```
# create stacked bar chart (same visualization logic as first chart)
fig, ax = plt.subplots(figsize=(12, 6))

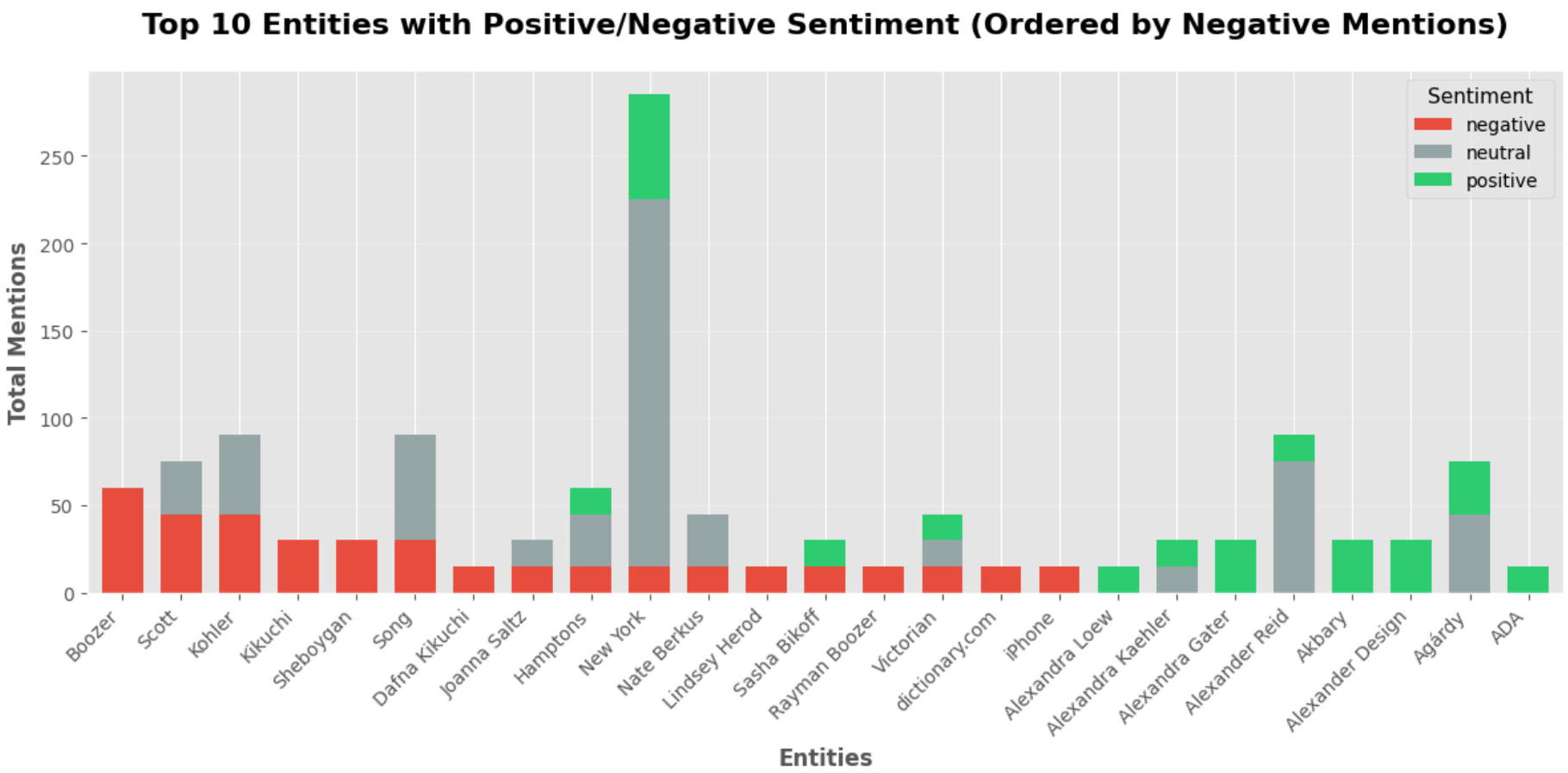
pivot_entity_negative.plot(kind='bar',
                           stacked=True,
                           ax=ax,
                           color=[colors[col] for col in pivot_entity_negative.columns],
                           width=0.7)

# Customize the chart
ax.set_title('Top 10 Entities with Positive/Negative Sentiment (Ordered by Negative Mentions)',
             fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Entities', fontsize=12, fontweight='bold')
ax.set_ylabel('Total Mentions', fontsize=12, fontweight='bold')
ax.legend(title='Sentiment', title_fontsize=11, fontsize=10)
ax.set_xticklabels(pivot_entity_negative.index, rotation=45, ha='right')

# Add grid for better readability
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()

print("\nNote: Entities are ordered left-to-right by negative mention count (highest first).")
```




```
Note: Entities are ordered left-to-right by negative mention count (highest first).
```

Observation

This graph brings it all together. The entities with the highest negative sentiment should appear on the far left, decreasing to the right, where entities have more neutral and/or positive sentiment. A client might use this graph to determine which entities to avoid most, while also identifying which entities have the highest negative sentiment with the least positive or neutral sentiment.

This prioritization helps e-commerce clients quickly identify high-risk entities while understanding their full sentiment context. An entity with 100 negative mentions and 200 positive mentions tells a different story than one with 100 negative mentions and 0 positive mentions. Both are shown here, but they are ordered by the strength of the negative signal.

Analyzing Confidence Scores for Negative Sentiment

WigglNET uses RoBERTa, which provides confidence scores (0-1) indicating how certain the model is about each sentiment classification.

Examining confidence alongside negative mention counts distinguishes reliable warning signals from potentially misclassified or ambiguous entities.

The same approach can be applied to positive sentiment to surface partnership candidates, but it is omitted here for brevity. The methodology is identical, with the color map and sort field swapped.

Reusable Visualization Functions

In [11]:
```
import matplotlib.cm as cm
from matplotlib.colors import Normalize

def plot_confidence_bar_chart(data, mentions_col, confidence_col,
                              sentiment_type, color_map='Reds'):
    # sort by mentions for better visualization
    data_sorted = data.sort_values(mentions_col, ascending=True)

    # create figure object and ax
    fig, ax = plt.subplots(figsize=(12, 10))

    # normalize the confidence scores across the data,
    #   so that it can be color mapped
    norm = Normalize(vmin=data_sorted[confidence_col].min(),
                     vmax=data_sorted[confidence_col].max())
    cmap = cm.get_cmap(color_map) # call in color map

    # Create bars with color based on confidence
    ax.barh(range(len(data_sorted)),
            data_sorted[mentions_col],
            color=cmap(norm(data_sorted[confidence_col])))

    # Customize chart
    ax.set_yticks(range(len(data_sorted)))
    ax.set_yticklabels(data_sorted['ENTITY_TEXT'], fontsize=10)
    ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
    ax.set_ylabel('Entity', fontsize=12, fontweight='bold')
    ax.set_title(f'Top 25 {sentiment_type}ly Mentioned Entities\n(Color Intensity = Confidence Level)',
                 fontsize=14, fontweight='bold', pad=20)

    # Add grid
    ax.xaxis.grid(True, alpha=0.3)
    ax.set_axisbelow(True)

    # Add colorbar
    sm = cm.ScalarMappable(cmap=cmap, norm=norm)
    sm.set_array([])
    cbar = plt.colorbar(sm, ax=ax)
    cbar.set_label('Average Confidence Score', fontsize=11, fontweight='bold')

    # Add value labels on bars
    for i, (mentions, confidence) in enumerate(zip(data_sorted[mentions_col],
                                                   data_sorted[confidence_col])):
        ax.text(mentions + 1, i, f'{int(mentions)} ({confidence:.2f})',
                va='center', fontsize=8, color='black')

    plt.tight_layout()
    plt.show()
```

In [12]:
```
def plot_confidence_scatter(data, mentions_col, confidence_col,
                            sentiment_type, color_map='Reds',
                            quadrant_color_high='#ffcccc', quadrant_color_low='#ffffcc',
                            max_mentions=None):
    # Create figure
    fig, ax = plt.subplots(figsize=(14, 8))

    # Plot scatter points
    scatter = ax.scatter(data[mentions_col],
                         data[confidence_col],
                         s=200,
                         c=data[confidence_col],
                         cmap=color_map,
                         alpha=0.6,
                         edgecolors='black',
                         linewidth=1.5)

    # Add entity labels
    for idx, row in data.iterrows():
        ax.annotate(row['ENTITY_TEXT'],
                    (row[mentions_col], row[confidence_col]),
                    fontsize=9,
                    xytext=(5, 5),
                    textcoords='offset points',
                    bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.3))

    # Add quadrant lines (median splits)
    median_mentions = data[mentions_col].median()
```

```
median_confidence = data[confidence_col].median()

ax.axvline(median_mentions, color='gray', linestyle='--', alpha=0.5, linewidth=1)
ax.axhline(median_confidence, color='gray', linestyle='--', alpha=0.5, linewidth=1)

# Add quadrant labels
ax.text(median_mentions * 1.5, data[confidence_col].max() * 0.98,
        'High Confidence\nHigh Volume', ha='center', fontsize=10,
        bbox=dict(boxstyle='round', facecolor=quadrant_color_high, alpha=0.7))
ax.text(median_mentions * 0.5, data[confidence_col].max() * 0.98,
        'High Confidence\nLow Volume', ha='center', fontsize=10,
        bbox=dict(boxstyle='round', facecolor=quadrant_color_low, alpha=0.7))

# Customize chart
ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
ax.set_ylabel('Average Confidence Score', fontsize=12, fontweight='bold')
ax.set_title(f'Confidence vs. {sentiment_type} Mention Count for Top 25 Entities',
             fontsize=14, fontweight='bold', pad=20)
ax.grid(True, alpha=0.3)

# Cap the x-axis if max_mentions is specified
if max_mentions is not None:
    ax.set_xlim(0, max_mentions)

# Add colorbar
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Confidence Score', fontsize=11, fontweight='bold')

    plt.tight_layout()
    plt.show()
```

Filter negative sentiment

In [13]:
```
# Filter for negative sentiment only
df_negative = df[df['SENTIMENT'] == 'negative'].copy()

print(f"Found {len(df_negative)} negative sentiment records")
print(f"Confidence score range: {df_negative['CONFIDENCE'].min():.3f} to {df_negative['CONFIDENCE'].max():.3f}")
print(f"Average confidence: {df_negative['CONFIDENCE'].mean():.3f}")
```

```
Found 255 negative sentiment records
Confidence score range: 0.519 to 0.825
Average confidence: 0.674
```

In [14]:
```
# Aggregate negative entities: sum mentions and average confidence
negative_entity_stats = df_negative.groupby('ENTITY_TEXT').agg({
    'MENTIONS': 'sum',
    'CONFIDENCE': 'mean'
}).reset_index()

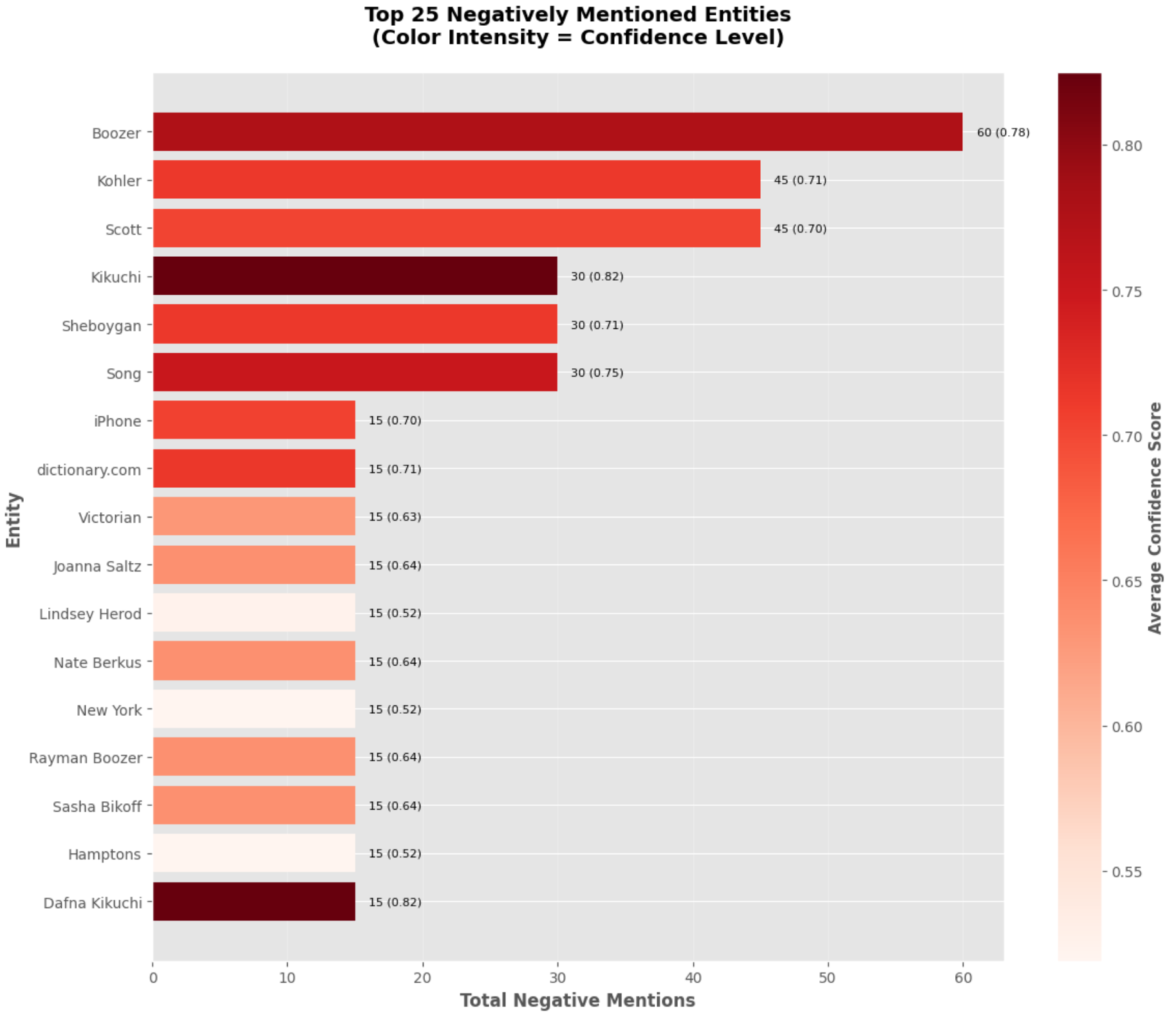
negative_entity_stats.columns = ['ENTITY_TEXT', 'TOTAL_NEGATIVE_MENTIONS', 'AVG_CONFIDENCE']
top_25_negative = negative_entity_stats.nlargest(25, 'TOTAL_NEGATIVE_MENTIONS')
top_25_negative['AVG_CONFIDENCE'] = top_25_negative['AVG_CONFIDENCE'].round(3)
```

Bar Chart: Color Gradient by Confidence

Bar height represents negative mention count, while color intensity shows confidence level. Darker bars indicate higher model confidence in the negative sentiment classification.

In [15]:
```
# Use the reusable function for negative entities bar chart
plot_confidence_bar_chart(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds'
)
```

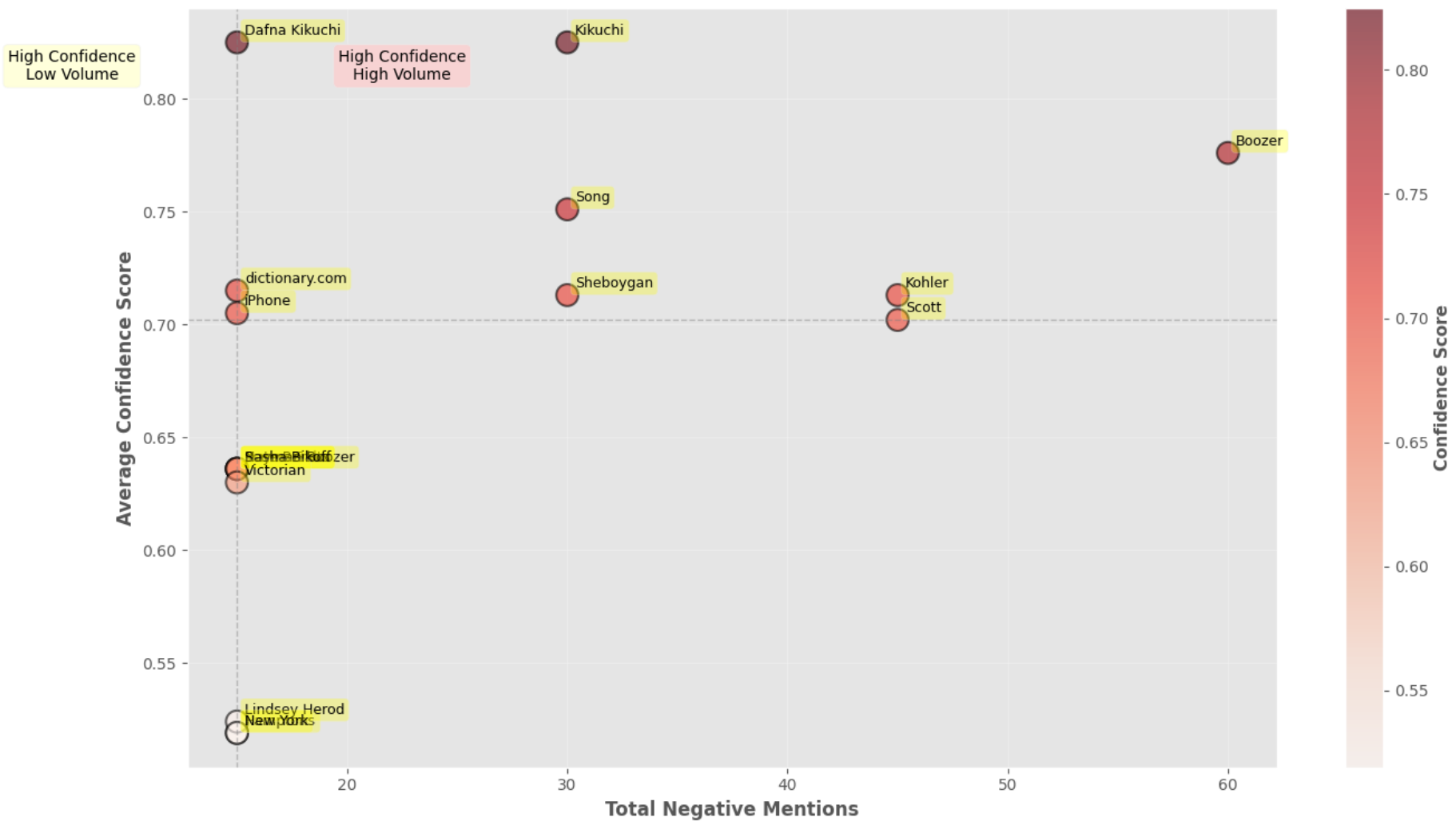
```
/tmp/ipykernel_9922/2958685450.py:16: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotl
ib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``p
yplot.get_cmap()`` instead.
  cmap = cm.get_cmap(color_map) # call in color map
```



Scatter Plot: Confidence vs. Negative Mentions

The top-right quadrant (high mentions, high confidence) represents the most reliable warning signs. High-confidence/low-mention entities are emerging concerns, high-mention/low-confidence entities may be misclassifications worth investigating, and low/low entities are the least concerning.

In [16]:
```
# Use the reusable function for negative entities scatter plot
plot_confidence_scatter(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds',
    quadrant_color_high='#ffcccc',
    quadrant_color_low='#ffffcc'
)
```



As mentioned before, entities on the far right side of the scatter plot have higher mentions, while those in the upper-right area have high confidence scores.

These are what the different confidence thresholds mean:

(>0.85) - the RoBERTa model is very certain about the sentiment classification
(0.70-0.85) - the model is moderately certain, which is typical for clear negative language
(<0.70) - low confidence: uncertain classification. May indicate ambiguous language or misclassification

For WigglNET clients, high-confidence, high-volume entities will appear as darker, taller bars, indicating the most reliable negative signals. E-commerce sellers should investigate these entities thoroughly before associating with them.

This two-dimensional analysis helps clients distinguish between solid negative signals and potentially uncertain classifications, leading to better-informed business decisions.

Milestone 1 Conclusion

The best way for a seller or manufacturer to understand what is happening in a market is to look directly at what is being discussed. Reading articles one by one and judging whether an entity is positive or negative is inefficient compared to the classification models WigglNET runs to identify the sentiment of keywords and entities across hundreds of articles. This is not to say that manually applying sentiment labels is useless. To develop ground-truth labels for a client's particular market needs, there must be human-made truths that the model can compare its predictions to. From there, the model can improve its predictions based on outcomes, results, and KPIs from the client's analytics.

This notebook reviewed a process for validating results produced by RoBERTa to verify the effectiveness of the model's labeling for a particular target market. It was visualized in three stacked bar charts showing different ways clients could use this data to make decisions about topics with positive and negative sentiment.

Milestone 2: Data Preparation

Next in the pipeline is preparing the data for machine learning model building and evaluation. In the following sections, the data will undergo cleaning, extraction, selection, transformation, and feature engineering.

Drop any features that are not useful for your model building and explain why they are not useful.

In [17]:
```
df = df.drop(columns=["ARTICLE_ID", "CONFIDENCE"])
```

For model development, the only columns that need to be dropped are `ARTICLE_ID` and `CONFIDENCE`. Article ID is set automatically by the program creating the data, and Confidence is RoBERTa's confidence in the sentiment label it produced. If this field were fed into a classifier validating the accuracy of the labels, the model would look extremely accurate under false pretenses because the feature it relies on is derived from the very label it is trying to validate.

Check for missing data

In [18]:
```
# Sum the NaNs
df.isnull().sum()
```

Out[18]:
```
TITLE             0
PUBLISHED_DATE    0
KEYWORD_TEXT      0
ENTITY_TEXT       0
ENTITY_TYPE       0
SENTIMENT         0
MENTIONS          0
dtype: int64
```

A check using `df.isnull().sum()` returned zero null values across all columns. This is expected: the data was queried from a database populated by a clean upstream pipeline, and the SQL query explicitly filtered out rows where PUBLISHED_DATE IS NOT NULL. No imputation or row removal was required.

Engineering new features: Published Date

From the `PUBLISHED_DATE` column, four features will be created:

`PUB_YEAR` : Year the article was published
`PUB_MONTH` : Month the article was published
`PUB_DOW` : Day of the week the article was published

In [19]:
```
# converting to datetime if not already
df['PUBLISHED_DATE'] = pd.to_datetime(df['PUBLISHED_DATE'])

# adding the three new features
df['pub_year']       = df['PUBLISHED_DATE'].dt.year
df['pub_month']      = df['PUBLISHED_DATE'].dt.month
df['pub_dow']        = df['PUBLISHED_DATE'].dt.dayofweek   # 0=Monday, 6=Sunday

# clean up by dropping PUBLISHED_DATE
df = df.drop(columns=['PUBLISHED_DATE'])
```

Feature transformations: Mentions

In [20]:
```
# Imports and preprocessing
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler, TargetEncoder
```

`MENTIONS` is skewed because a few entities are mentioned hundreds of times. Applying a log transformation compresses the range and helps linear models. The histogram below shows the current distribution of the mentions field, which is right-skewed.

In [21]:
```
# pick a representative spread of values from your data
# Sample representative values from the actual distribution
sample_values = np.unique(
    np.percentile(df['MENTIONS'], [10, 25, 50, 75, 90, 95, 100]).astype(int)
)
log_values = np.log1p(sample_values)

fig, axes = plt.subplots(2, 1, figsize=(14, 5))

# === Top: raw MENTIONS number line ===
axes[0].hlines(0, 0, 40, colors='black', linewidth=1)
axes[0].scatter(sample_values, [0]*len(sample_values),
                color='#e74c3c', s=120, zorder=3, edgecolor='black')

# Label each point with its value
for v in sample_values:
    axes[0].annotate(str(v), (v, 0), textcoords='offset points',
                     xytext=(0, 12), ha='center', fontsize=10)

axes[0].set_xlim(-1, 41)
axes[0].set_ylim(-1, 1)
axes[0].set_title('Before: Raw MENTIONS', fontsize=13, fontweight='bold', loc='left')
axes[0].set_yticks([])
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)
axes[0].spines['left'].set_visible(False)

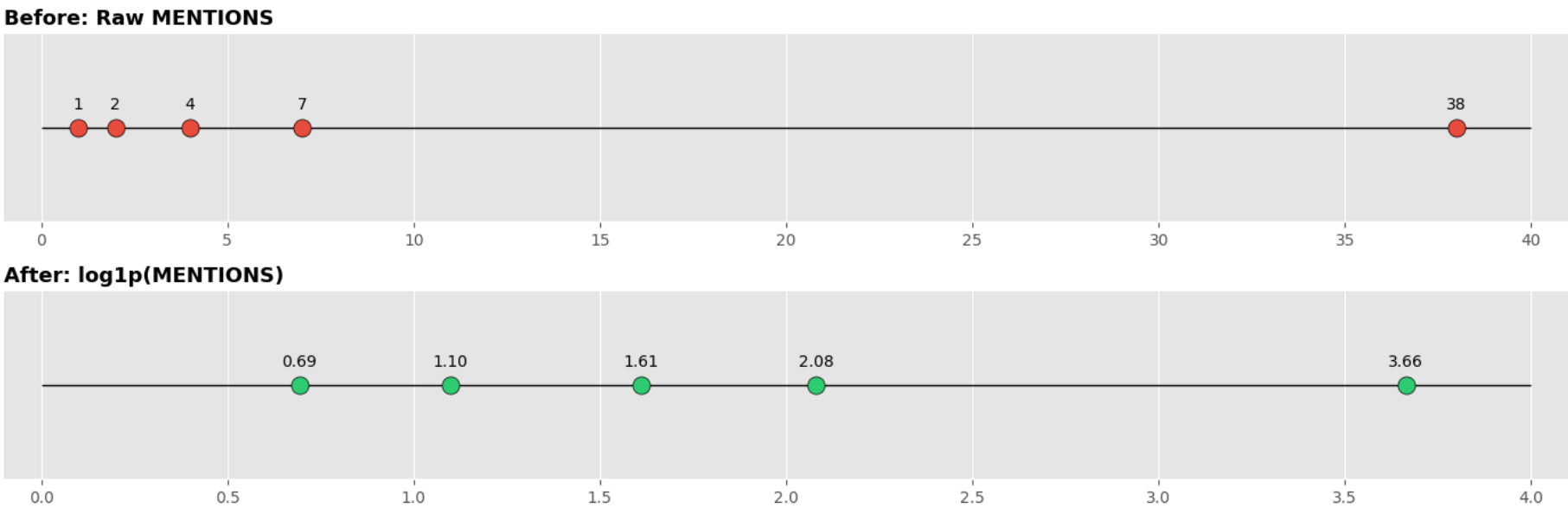
# === Bottom: log-transformed number line ===
axes[1].hlines(0, 0, 4, colors='black', linewidth=1)
axes[1].scatter(log_values, [0]*len(log_values),
```

```
                color='#2ecc71', s=120, zorder=3, edgecolor='black')

for raw, log_v in zip(sample_values, log_values):
    axes[1].annotate(f'{log_v:.2f}', (log_v, 0), textcoords='offset points',
                     xytext=(0, 12), ha='center', fontsize=10)

axes[1].set_xlim(-0.1, 4.1)
axes[1].set_ylim(-1, 1)
axes[1].set_title('After: log1p(MENTIONS)', fontsize=13, fontweight='bold', loc='left')
axes[1].set_yticks([])
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)
axes[1].spines['left'].set_visible(False)

plt.suptitle('Redistribute Spacing of MENTIONS',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



Creating a before-and-after comparison of the logarithm transformation makes it look as if not much changed. However, the transformation compressed the long right tail.

In the raw data, a row with 38 mentions sat 37 units away from a row with 1 mention, a distance the model would treat as enormous. After log1p, that same gap shrinks to about 3 units, putting high- and low-mention rows on a comparable scale. The visual shape does not change dramatically because most rows already cluster at 1-3 mentions, but the underlying distance relationships between values are now proportional, which is what linear models actually need.

In [22]:
```
# Apply the log transformation to MENTIONS before splitting the data
df['MENTIONS'] = np.log1p(df['MENTIONS'])

print(df['MENTIONS'].describe())
```

```
count    19367.000000
mean         1.045827
std          0.474284
min          0.693147
25%          0.693147
50%          0.693147
75%          1.098612
max          3.663562
Name: MENTIONS, dtype: float64
```

Separating the Target from Predictor Variables

To keep the model from accidentally learning from the answer column, `SENTIMENT` is separated into `y`, and the remaining columns are stored in `X` as predictor variables.

In [23]:
```
X = df.drop(columns=['SENTIMENT'])
y = df['SENTIMENT']

print('Predictor shape:', X.shape)
print('Target shape:', y.shape)
print('Target classes:')
print(y.value_counts())
```

```
Predictor shape: (19367, 8)
Target shape: (19367,)
Target classes:
SENTIMENT
neutral     11890
positive     7222
negative      255
Name: count, dtype: int64
```

Performing the Train/Test Split

The train/test split is performed before target encoding because target encoding uses `y` to calculate category values. The test set must stay unseen until evaluation.

In [24]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print('X_train shape:', X_train.shape)
print('X_test shape:', X_test.shape)
print('y_train shape:', y_train.shape)
print('y_test shape:', y_test.shape)
```

```
X_train shape: (15493, 8)
X_test shape: (3874, 8)
y_train shape: (15493,)
y_test shape: (3874,)
```

Target Encoding: Text and Keywords

`ENTITY_TEXT` and `KEYWORD_TEXT` cannot be read by a machine learning model as text. Target encoding converts each category to a numeric value based on the target values in the training data. To avoid data leakage, the encoder is fit only on `X_train` and `y_train`, then applied to both `X_train` and `X_test`.

In [25]:
```
sentiment_map = {
    'negative': 0,
    'neutral': 1,
    'positive': 2
}

y_train_encoded = y_train.map(sentiment_map)

target_encoder = TargetEncoder(target_type='continuous', random_state=42)

train_target_encoded = target_encoder.fit_transform(
    X_train[['ENTITY_TEXT', 'KEYWORD_TEXT']],
    y_train_encoded
)

test_target_encoded = target_encoder.transform(
    X_test[['ENTITY_TEXT', 'KEYWORD_TEXT']]
)

train_target_encoded = pd.DataFrame(
    train_target_encoded,
    columns=['ENTITY_ENC', 'KEYWORD_ENC'],
    index=X_train.index
)

test_target_encoded = pd.DataFrame(
    test_target_encoded,
    columns=['ENTITY_ENC', 'KEYWORD_ENC'],
    index=X_test.index
)

X_train = X_train.drop(columns=['ENTITY_TEXT', 'KEYWORD_TEXT']).join(train_target_encoded)
X_test = X_test.drop(columns=['ENTITY_TEXT', 'KEYWORD_TEXT']).join(test_target_encoded)

print('After target encoding:')
print(X_train[['ENTITY_ENC', 'KEYWORD_ENC']].describe())
```

```
After target encoding:
         ENTITY_ENC   KEYWORD_ENC
count  15493.000000  15493.000000
mean       1.359563      1.357765
std        0.440269      0.154827
min        0.000000      0.000000
25%        1.000000      1.318420
50%        1.091779      1.354844
75%        2.000000      1.404257
max        2.000000      2.000000
```

TF-IDF Encoding: Title

`TfidfVectorizer` is fit on the training titles only. The fitted vectorizer is then used to transform both the training and test titles so the same title vocabulary is used in both datasets.

In [26]:
```
tfidf = TfidfVectorizer(
    max_features=100,
    stop_words='english',
```

```
    ngram_range=(1, 2)
)

train_title_matrix = tfidf.fit_transform(X_train['TITLE'])
test_title_matrix = tfidf.transform(X_test['TITLE'])

title_columns = [f'title_{w}' for w in tfidf.get_feature_names_out()]

train_title_df = pd.DataFrame(
    train_title_matrix.toarray(),
    columns=title_columns,
    index=X_train.index
)

test_title_df = pd.DataFrame(
    test_title_matrix.toarray(),
    columns=title_columns,
    index=X_test.index
)

X_train = X_train.drop(columns=['TITLE']).join(train_title_df)
X_test = X_test.drop(columns=['TITLE']).join(test_title_df)

print(f'After TF-IDF, added {train_title_df.shape[1]} title-derived columns')
print('Sample title features:', title_columns[:10])
```

```
After TF-IDF, added 100 title-derived columns
Sample title features: ['title_2026', 'title_2026 color', 'title_30', 'title_35', 'title_36', 'title_45', 'title_aly
ssa', 'title_alyssa khan', 'title_announces', 'title_announces 2026']
```

One Hot Encoding: Entity Type

`ENTITY_TYPE` has no meaningful ranking, so one-hot encoding converts it to dummy variables. The test set is aligned to the training dummy columns so both datasets have the same feature layout.

In [27]:
```
train_entity_type_dummies = pd.get_dummies(X_train['ENTITY_TYPE'], prefix='etype', dtype=int)
test_entity_type_dummies = pd.get_dummies(X_test['ENTITY_TYPE'], prefix='etype', dtype=int)

test_entity_type_dummies = test_entity_type_dummies.reindex(
    columns=train_entity_type_dummies.columns,
    fill_value=0
)

X_train = X_train.drop(columns=['ENTITY_TYPE']).join(train_entity_type_dummies)
X_test = X_test.drop(columns=['ENTITY_TYPE']).join(test_entity_type_dummies)

print('After one-hot encoding ENTITY_TYPE:')
print(X_train.filter(like='etype_').head())
```

```
After one-hot encoding ENTITY_TYPE:
       etype_GPE  etype_NORP  etype_ORG  etype_PERSON  etype_PRODUCT
951            1           0          0             0              0
8583           0           0          1             0              0
1767           0           0          0             1              0
18939          1           0          0             0              0
6539           1           0          0             0              0
```

Ready for Modeling

All feature columns are now numeric and were prepared using the training data first. `SENTIMENT` remains separate as the target variable.

In [28]:
```
print('Training feature types:')
print(X_train.dtypes.value_counts())

print('Testing feature types:')
print(X_test.dtypes.value_counts())
```

```
Training feature types:
float64    103
int64        5
int32        3
Name: count, dtype: int64
Testing feature types:
float64    103
int64        5
int32        3
Name: count, dtype: int64
```

Quick Overview of the Data

Below are the final results of the training data after model preparation.

In [29]:
```
X_train
```

Out[29]:

	MENTIONS	pub_year	pub_month	pub_dow	ENTITY_ENC	KEYWORD_ENC	title_2026	title_2026 color	title_30	title_35	...	title_va
951	1.791759	2026	3	2	1.094183	1.360171	0.0	0.0	0.000000	0.0	...	
8583	1.386294	2025	12	1	1.058917	1.524817	0.0	0.0	0.000000	0.0	...	
1767	0.693147	2026	3	2	1.000000	1.496808	0.0	0.0	0.000000	0.0	...	
18939	0.693147	2022	4	4	0.000000	1.157574	0.0	0.0	0.000000	0.0	...	
6539	1.386294	2026	1	1	1.000000	1.524817	0.0	0.0	0.000000	0.0	...	
...	...	...	...	...	...	...	...	...	...	...	...	
11236	1.791759	2025	9	1	1.414740	1.346355	0.0	0.0	0.000000	0.0	...	
17896	2.197225	2022	11	1	2.000000	1.090017	0.0	0.0	0.000000	0.0	...	
9077	1.098612	2025	11	0	1.000000	1.353867	0.0	0.0	0.000000	0.0	...	
16995	0.693147	2023	8	2	1.000000	1.398085	0.0	0.0	0.000000	0.0	...	
7296	1.098612	2025	12	1	1.000000	1.329395	0.0	0.0	0.609176	0.0	...	

15493 rows × 111 columns

Milestone 3: Model Building and Evaluation

Classification Model: Logistic Regression

Logistic regression is used as a baseline classification model. The model is trained only on `X_train` and `y_train`, then evaluated on the unseen `X_test` rows.

In [30]:
```
log_reg_model = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=2000))
])

log_reg_model.fit(X_train, y_train)
y_pred = log_reg_model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Classification report:')
print(classification_report(y_test, y_pred))
print('Confusion matrix:')
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.9217862674238513
Classification report:
              precision    recall  f1-score   support

    negative       0.92      0.94      0.93        51
     neutral       0.93      0.94      0.94      2378
    positive       0.90      0.89      0.90      1445

    accuracy                           0.92      3874
   macro avg       0.92      0.92      0.92      3874
weighted avg       0.92      0.92      0.92      3874

Confusion matrix:
[[  48    3    0]
 [   4 2231  143]
 [   0  153 1292]]
```

The model predicted the RoBERTa-generated sentiment labels correctly 92.17% of the time, which is **not** the same as proving that RoBERTa's real-world effectiveness was 92.17%.

GridSearchCV with Pipeline Classifiers

After establishing logistic regression as a baseline, `GridSearchCV` can compare multiple classifier algorithms. Each model is placed inside a `Pipeline`, which keeps preprocessing and model training organized. The grid search is fit only on `X_train` and `y_train`; the held-out `X_test` set is used only after the best model is selected.

The step below defines each model's settings before model selection occurs. `StandardScaler` is applied to models that benefit from standardization, while `MinMaxScaler` is used for KNN because KNN is distance-based and can be sensitive to feature scale. Hyperparameters are set for each model so `GridSearchCV` can select the strongest configuration.

In [31]:
```
# The candidate hyperparameter values are set before training,
# and GridSearchCV selects the best-performing values through cross-validation.
model_configs = {
    # Set the baseline model
    'Logistic Regression': {
        'pipeline': Pipeline(steps=[
            ('scaler', StandardScaler()),  # StandardScaler used for baseline model
```

```
        ('classifier', LogisticRegression(max_iter=2000))
    ]),
    'params': {  # GridSearchCV tests these combinations
        'classifier__C': np.logspace(-2, 2, 5),
        'classifier__penalty': ['l2']
    }
},

# Compare with KNN
# KNN uses distances between observations to classify new records.
# Because distance-based models are sensitive to feature scale,
# MinMaxScaler places numeric predictors on a common 0-to-1 scale
# before GridSearchCV searches for the best K value.
'KNN': {
    'pipeline': Pipeline(steps=[
        ('scaler', MinMaxScaler()),
        ('classifier', KNeighborsClassifier())
    ]),
    'params': {  # GridSearchCV tests these combinations
        'classifier__n_neighbors': [3, 5, 7, 9, 11, 15, 21],
        'classifier__weights': ['uniform', 'distance']
    }
},

# Compare with Random Forest
# Random Forest classifiers do not require feature scaling because
# decision trees make splits based on feature thresholds instead of
# distance calculations.
    'Random Forest': {
        'pipeline': Pipeline(steps=[
            ('classifier', RandomForestClassifier(random_state=42, n_jobs=-1))
        ]),
        'params': {  # GridSearchCV tests these combinations
            'classifier__n_estimators': [100, 200],
            'classifier__max_depth': [None, 10, 20]
        }
    },

    # Compare with support vector machine
    # SVM is sensitive to feature scaling, so StandardScaler is used to
    # place features on a comparable mean/std scale.
    'Linear SVM': {
        'pipeline': Pipeline(steps=[
            ('scaler', StandardScaler()),
            ('classifier', LinearSVC(max_iter=5000, random_state=42))
        ]),
        'params': {  # GridSearchCV tests these combinations
            'classifier__C': [0.1, 1, 10]
        }
    }
}
```

Model Selection Results

A list of each model's best weighted CV scores is generated to determine which model performed best. This process may take about two to five minutes, depending on processing power.

In [32]:
```
import warnings
from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=ConvergenceWarning)
warnings.filterwarnings('ignore', message='.*sklearn.utils.parallel.delayed.*', category=UserWarning)

grid_results = []
best_estimators = {}

# For each model in the configuration it was given,
for model_name, config in model_configs.items():
    # Apply GridSearch
    grid_search = GridSearchCV(
        estimator=config['pipeline'],  # define the pipeline
        param_grid=config['params'],  # define its parameters
        cv=3,
        scoring='f1_weighted',
        n_jobs=-1
    )

    grid_search.fit(X_train, y_train)

    grid_results.append({
        'Model': model_name,
        'Best CV F1 Weighted': grid_search.best_score_,
        'Best Parameters': grid_search.best_params_
    })
```

```
    best_estimators[model_name] = grid_search.best_estimator_

grid_results_df = pd.DataFrame(grid_results).sort_values(
    by='Best CV F1 Weighted',
    ascending=False
)

grid_results_df
```

Out[32]:

	Model	Best CV F1 Weighted	Best Parameters
2	Random Forest	0.997482	{'classifier__max_depth': None, 'classifier__n...
1	KNN	0.996579	{'classifier__n_neighbors': 3, 'classifier__we...
0	Logistic Regression	0.926247	{'classifier__C': 100.0, 'classifier__penalty'...
3	Linear SVM	0.921996	{'classifier__C': 10}

Evaluate the Best Grid Search Model

The model with the highest cross-validation score is selected from the grid search results, then evaluated once on `X_test`.

In [33]:
```
best_model_name = grid_results_df.iloc[0]['Model']
best_grid_model = best_estimators[best_model_name]

y_pred_grid = best_grid_model.predict(X_test)

print('Best model:', best_model_name)
print('Test Accuracy:', accuracy_score(y_test, y_pred_grid))
print('Classification report:')
print(classification_report(y_test, y_pred_grid))
print('Confusion matrix:')
print(confusion_matrix(y_test, y_pred_grid))
```

```
Best model: Random Forest
Test Accuracy: 0.9994837377387713
Classification report:
              precision    recall  f1-score   support

    negative       1.00      1.00      1.00        51
     neutral       1.00      1.00      1.00      2378
    positive       1.00      1.00      1.00      1445

    accuracy                           1.00      3874
   macro avg       1.00      1.00      1.00      3874
weighted avg       1.00      1.00      1.00      3874

Confusion matrix:
[[  51    0    0]
 [   0 2378    0]
 [   0    2 1443]]
```

Random Forest achieved a test accuracy of 99.95% and the highest cross-validation weighted F1-score of 0.9975. This means it was highly successful at reproducing the RoBERTa-generated sentiment labels in the held-out test set.

Before GridSearchCV, the baseline logistic regression model scored 92.17%. After model selection and hyperparameter tuning, the model increased accuracy by 7.78 percentage points.

Milestone 3 Summary

In Milestone 3, after separating the target, `y`, from the predictors, `X`, a baseline logistic regression algorithm was built to compare Random Forest, K-Nearest Neighbor, and Support Vector Machines against the baseline. To speed up hyperparameter tuning, GridSearchCV was used to find the best hyperparameters for each model. When the cross-validation scores were compared, Random Forest had the highest CV score at 99.7%. This was reflected in the evaluation, where Random Forest was chosen as the best model for predicting the effectiveness of the RoBERTa model's target labeling.

Because this is a multiclass classification problem with uneven class support across negative, neutral, and positive sentiment labels, weighted F1-score was used during model selection. Weighted F1 balances precision and recall while accounting for the number of observations in each class, making it more informative than accuracy alone when classes are imbalanced. Accuracy is still reported for readability, but weighted F1 is the primary model-selection metric.